Unit 3
Programming Fundamentals

Python as a first high-level language · IDE · operators · control flow

VARIABLES & TYPES | INPUT / PRINT | IF / ELIF / ELSE | FOR / WHILE | LIBRARIES & DEBUG

Type every snippet. Reading Python is not the same as making it run.

0. Computers are obedient, not clever

A computer does nothing until a **program** — a list of instructions — tells it what to do. A **programming language** is the agreed vocabulary and grammar for those instructions. Python is a **high-level** language: it looks like structured English, not like raw `0`s and `1`s.

LOW-LEVEL
Binary machine code. Fast for the chip, painful for humans.

MID-LEVEL
Assembly or C-like languages: mnemonics plus some hardware control. C++ sits near this border and is used for systems and games.

HIGH-LEVEL
Python, Java, C++. Designed for people. Java's motto is "write once, run anywhere." Python is the course language because it is readable and has a huge library world.

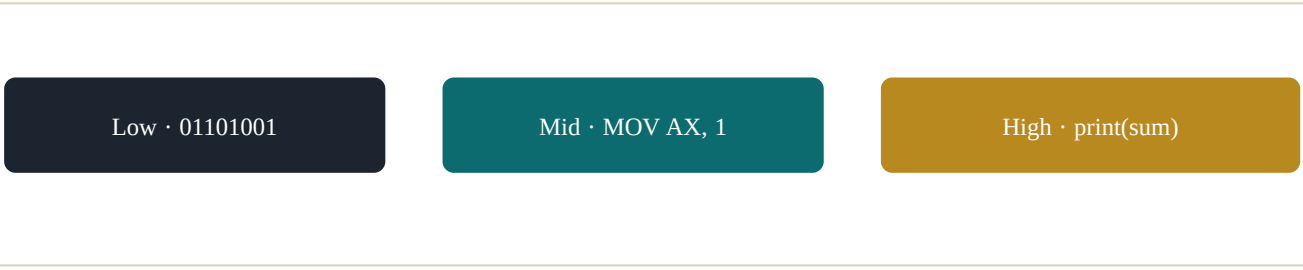


Figure 1. Closer to the machine on the left, closer to human language on the right.

CAREERS Python pays the rent in web work (Django, Flask), data science, AI, and automation. This unit is not "toy coding"; it is the on-ramp those fields share.

1. The workshop — VS Code and a first program

An **IDE** (Integrated Development Environment) is an editor plus runner plus debugger. Visual Studio Code is the recommended desktop IDE. Online runners such as online-python.com are acceptable when a lab PC is locked down.

1. Install VS Code. Install the Python extension. Confirm `python --version` in the terminal.
2. Save a file as `hello.py`. Run it with the play button or `python hello.py`.
3. Read the terminal. That window is part of the program, not decoration.

2. Variables, types, input, output

A **variable** is a named box. Names: letters, digits, underscore; must start with a letter or `_`; case-sensitive; cannot be a keyword (`if`, `for`, `class`…). Python infers the type from the value you put in the box.

Table 1. Types you will actually use this year

Type	Example	Meaning
`int`	`18`	Whole number
`float`	`1.75`	Decimal
`str`	`"Ali"`	Text
`bool`	`True`	Boolean (note the capital)
`list` / `tuple` / `dict` / `set`	`[1,2]`	Collections (Class XII deepens these)

```
name = input("Enter your name: ")
age = int(input("Enter your age: "))
height = float(input("Enter height (m): "))
print(f"My name is {name} and I am {age} years old.")
```

WATCH `input()` always returns a **string**. Adding `"3" + "4"` gives `"34"`. Wrap with `int()` or `float()` before any arithmetic.

3. Operators — the verbs of an expression

ARITHMETIC
`+ - * / // % **`
true divide `/`, floor divide `//`, remainder `%`, power `**`

ASSIGNMENT
`= += -= *= /= //= %= **=`

RELATIONAL
`== != > < >= <=` → `bool`

LOGICAL
`and` `or` `not` — same truth tables as Unit 1 AND/OR/NOT

MEMBERSHIP
`in` `not in`

BITWISE
`& | ^ ~ << >>` (bit-level AND/OR/XOR)

$$4 == 5 - 1 \Rightarrow \textbf{True}, \quad 55 \geq 11 \times 5 \Rightarrow \textbf{True}, \quad 22 \neq 23 - 1 \Rightarrow \textbf{False}$$

4. Control structures — sequence, selection, repetition

Almost every program is assembled from three shapes.

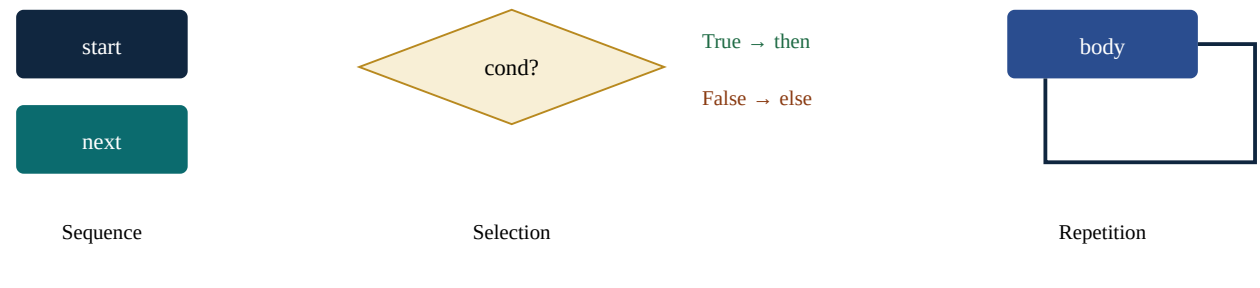


Figure 2. The three control-flow shapes. Indentation in Python *is* the shape.

4.1 Sequence

Statements run top to bottom. Default behaviour.

```
print("Enter two numbers")
a = int(input("Enter first number: "))
b = int(input("Enter second number: "))
total = a + b
print("Sum =", total)
```

4.2 Selection — `if`, `if-else`, `elif`

A colon ends the condition. The following indented block runs only when the condition is true. A lone `if` may do nothing; `if-else` always runs exactly one of the two blocks.

```
age = int(input("Enter your age: "))
if age >= 18:
    print("You are an Adult.")
    print("You are eligible to vote.")
elif age >= 13:
    print("Teenager — wait a few years to vote.")
else:
    print("Too young to vote.")
print("Program continues...")
```

4.3 Repetition — `for` and `while`

```
for i in range(5):        # 0,1,2,3,4
    print(i)
for i in range(2, 8):     # 2..7
    print(i)
for i in range(0, 10, 2): # step 2
    print(i)

count = 1
while count <= 5:
    print(count)
    count += 1            # forget this → infinite loop
```

`break` leaves the loop immediately. `continue` skips the rest of this iteration. Nested loops: the inner loop finishes a full cycle for every single outer step (this is why nested loops smell like $O(n^2)$).

$$\textbf{range}(a, b) = \{a, a+1, \ldots, b-1\}, \qquad \textbf{range}(a, b, s) \text{ steps by } s.$$

5. Libraries — do not reinvent the wheel

```
import math
import math as m
from math import sqrt, pi
# pip install numpy pandas matplotlib   # third-party
```

Table 2. Built-in libraries the textbook expects

Module	Useful names
`math`	`pi`, `sqrt`, `ceil`, `floor`, `pow`
`random`	`randint`, `random`, `uniform`
`datetime`	`datetime.now()`, `strftime`

6. Debugging — three families of pain

1. SYNTAX
The program never starts. Missing `)`, forgotten `:`, bad indent after `if`.

2. RUNTIME
Starts, then crashes. `10/0`, `int("abc")`. Read the traceback: type, file, line.

3. LOGICAL
Runs, prints the wrong answer. Classic: `i -= 1` instead of `i += 1` in a `while` — infinite loop that looks "alive".

Self-check

1. What does `print(3 + int("4"))` print? What about `print(3 + "4")`?
2. Rewrite "print all odd numbers from 1 to 15" with `range` and a step.
3. Name the error family: the program prints a negative average for a class of positive marks.

Answers. (1) `7`; the second raises `TypeError`. (2) `for i in range(1, 16, 2): print(i)`. (3) Logical — it ran.

Recap

1. Python is high-level; `input()` is always `str`.
2. Names: letter/underscore first; case matters; no keywords.
3. Operators: arithmetic, assignment, relational, logical, membership, bitwise.
4. Sequence / selection / repetition. Indentation defines blocks.
5. `range(a,b)` stops before b. `break` exits; `continue` skips.
6. Import `math`, `random`, `datetime`; install third-party with `pip`.
7. Debug by family: syntax, runtime, logical. Trust the traceback line number.